About incremental strategy

Incremental strategies for materializations optimize performance by defining how to handle new and changed data.

There are various strategies to implement the concept of incremental materializations. The value of each strategy depends on:

- The volume of data.
- The reliability of your `unique_key` .
- The support of certain features in your data platform.

An optional `incremental_strategy` config is provided in some adapters that controls the code that dbt uses to build incremental models.

> **MICROBATCH**
>
> The `microbatch` incremental strategy is intended for large time-series datasets. dbt will process the incremental model in multiple queries (or "batches") based on a configured `event_time` column. Depending on the volume and nature of your data, this can be more efficient and resilient than using a single query for adding new data.

Supported incremental strategies by adapter

This table shows the support of each incremental strategy across adapters available on dbt's Latest release track. Some strategies may be unavailable if you're not on **Latest** and the feature hasn't been released to the **Compatible** track.

If you're interested in an adapter available in dbt Core only, check out the adapter's individual configuration page for more details.

Click the name of the adapter in the following table for more information about supported incremental strategies:

Search table...

Data platform adapter	append	merge	delete+insert	insert_ov
dbt-postgres	✅	✅	✅	
dbt-redshift	✅	✅	✅	
dbt-bigquery		✅		
dbt-spark	✅	✅		
dbt-databricks	✅	✅	✅	
dbt-snowflake	✅	✅	✅	
dbt-trino	✅	✅	✅	
dbt-fabric	✅	✅	✅	
dbt-athena	✅	✅		
dbt-teradata	✅	✅	✅	

Configuring incremental strategy

The `incremental_strategy` config can either be defined in specific models or for all models in your `dbt_project.yml` file:

dbt_project.yml

```
models:
  +incremental_strategy: "insert_overwrite"
```

or:

models/my_model.sql

```
{{
  config(
    materialized='incremental',
    unique_key='date_day',
    incremental_strategy='delete+insert',
    ...
  )
}}

select ...
```

Strategy-specific configs

If you use the `merge` strategy and specify a `unique_key`, by default, dbt will entirely overwrite matched rows with new values.

On adapters which support the `merge` strategy, you may optionally pass a list of column names to a `merge_update_columns` config. In that case, dbt will update *only* the columns specified by the config, and keep the previous values of other columns.

models/my_model.sql

```
{{
  config(
    materialized = 'incremental',
    unique_key = 'id',
    merge_update_columns = ['email', 'ip_address'],
    ...
  )
}}

select ...
```

Alternatively, you can specify a list of columns to exclude from being updated by passing a list of column names to a `merge_exclude_columns` config.

models/my_model.sql

```
{{
  config(
    materialized = 'incremental',
    unique_key = 'id',
    merge_exclude_columns = ['created_at'],
    ...
```

```
  )
}}

select ...
```

About incremental_predicates

`incremental_predicates` is an advanced use of incremental models, where data volume is large enough to justify additional investments in performance. This config accepts a list of any valid SQL expression(s). dbt does not check the syntax of the SQL statements.

This an example of a model configuration in a `yml` file you might expect to see on Snowflake:

```
models:
  - name: my_incremental_model
    config:
      materialized: incremental
      unique_key: id
      # this will affect how the data is stored on disk, and indexed to limit
scans
      cluster_by: ['session_start']
      incremental_strategy: merge
      # this limits the scan of the existing table to the last 7 days of data
      incremental_predicates: ["DBT_INTERNAL_DEST.session_start >
dateadd(day, -7, current_date)"]
      # `incremental_predicates` accepts a list of SQL statements.
      # `DBT_INTERNAL_DEST` and `DBT_INTERNAL_SOURCE` are the standard
aliases for the target table and temporary table, respectively, during an
incremental run using the merge strategy.
```

Alternatively, here are the same configurations configured within a model file:

```
-- in models/my_incremental_model.sql

{{
  config(
    materialized = 'incremental',
    unique_key = 'id',
    cluster_by = ['session_start'],
    incremental_strategy = 'merge',
    incremental_predicates = [
      "DBT_INTERNAL_DEST.session_start > dateadd(day, -7, current_date)"
```

```
    ]
  )
}}

...
```

This will template (in the `dbt.log` file) a `merge` statement like:

```
merge into <existing_table> DBT_INTERNAL_DEST
    from <temp_table_with_new_records> DBT_INTERNAL_SOURCE
    on
        -- unique key
        DBT_INTERNAL_DEST.id = DBT_INTERNAL_SOURCE.id
        and
        -- custom predicate: limits data scan in the "old" data / existing
table
        DBT_INTERNAL_DEST.session_start > dateadd(day, -7, current_date)
    when matched then update ...
    when not matched then insert ...
```

Limit the data scan of *upstream* tables within the body of their incremental model SQL, which will limit the amount of "new" data processed/transformed.

```
with large_source_table as (

    select * from {{ ref('large_source_table') }}
    {% if is_incremental() %}
        where session_start >= dateadd(day, -3, current_date)
    {% endif %}

),

...
```

> **INFO**
>
> The syntax depends on how you configure your `incremental_strategy`:
>
> - If using the `merge` strategy, you may need to explicitly alias any columns with either `DBT_INTERNAL_DEST` ("old" data) or `DBT_INTERNAL_SOURCE` ("new" data).

- There's a decent amount of conceptual overlap with the `insert_overwrite` incremental strategy.

Built-in strategies

Before diving into custom strategies, it's important to understand the built-in incremental strategies in dbt and their corresponding macros:

Search table...

incremental_strategy	Corresponding macro
append	get_incremental_append_sql
delete+insert	get_incremental_delete_insert_sql
merge	get_incremental_merge_sql
insert_overwrite	get_incremental_insert_overwrite_sql
microbatch	get_incremental_microbatch_sql

For example, a built-in strategy for the `append` can be defined and used with the following files:

macros/append.sql

```
{% macro get_incremental_append_sql(arg_dict) %}

  {% do return(some_custom_macro_with_sql(arg_dict["target_relation"],
arg_dict["temp_relation"], arg_dict["unique_key"], arg_dict["dest_columns"],
arg_dict["incremental_predicates"])) %}

{% endmacro %}


{% macro some_custom_macro_with_sql(target_relation, temp_relation,
unique_key, dest_columns, incremental_predicates) %}

    {%- set dest_cols_csv = get_quoted_csv(dest_columns |
map(attribute="name")) -%}

    insert into {{ target_relation }} ({{ dest_cols_csv }})
    (
        select {{ dest_cols_csv }}
```

```
        from {{ temp_relation }}
    )

{% endmacro %}
```

Define a model models/my_model.sql:

```
{{ config(
    materialized="incremental",
    incremental_strategy="append",
) }}

select * from {{ ref("some_model") }}
```

About built-in incremental strategies

append

The `append` strategy is simple to implement and has low processing costs. It inserts selected records into the destination table without updating or deleting existing data. This strategy doesn't align directly with type 1 or type 2 slowly changing dimensions (SCD). It differs from SCD1, which overwrites existing records, and only loosely resembles SCD2. While it adds new rows (like SCD2), it doesn't manage versioning or track historical changes explicitly.

Importantly, `append` doesn't check for duplicates or verify whether a record already exists in the destination. If the same record appears multiple times in the source, it will be inserted again, potentially resulting in duplicate rows. This may not be an issue depending on your use case and data quality requirements.

delete+insert

The `delete+insert` strategy deletes the data for the `unique_key` from the target table and then inserts the data for those with a `unique_key`, which may be less efficient for larger datasets. It ensures updated records are fully replaced, avoiding partial updates and can be useful when a `unique_key` isn't truly unique or when `merge` is unsupported.

`delete+insert` doesn't map directly to SCD logic (type 1 or 2) because it overwrites data and tracks history.

For SCD2, use dbt snapshots, not `delete+insert`.

merge

`merge` inserts records with a `unique_key` that don't exist yet in the destination table and updates records with keys that do exist — mirroring the logic of SCD1, where changes are overwritten rather than historically tracked.

This strategy shouldn't be confused with `delete+insert` which deletes matching records before inserting new ones.

By specifying a `unique_key` (which can be composed of one or more columns), `merge` can also help resolve duplicates. If the `unique_key` already exists in the destination table, `merge` will update the record, so you won't have duplicates. If the records don't exist, `merge` will insert them.

Note, if you use `merge` without specifying a `unique_key`, it behaves like the `append` strategy.

While the `merge` strategy is useful for keeping tables current, it's best suited for smaller tables or incremental datasets. It can be expensive for large tables because it scans the entire destination table to determine what to update or insert.

insert_overwrite

The `insert_overwrite` strategy is used to efficiently update partitioned tables by replacing entire partitions with new data, rather than merging or updating individual rows. It overwrites only the affected partitions, not the whole table.

Because it is designed for partitioned data and replaces entire partitions wholesale, it does not align with typical SCD logic, which tracks row-level history or changes.

It's ideal for tables partitioned by date or another key and useful for refreshing recent or corrected data without full table rebuilds.

microbatch

`microbatch` is an incremental strategy designed for processing large time-series datasets by splitting the data into time-based batches (for example, daily or hourly). It supports parallel batch execution for faster runs.

For details on which incremental strategies are supported by each adapter, refer to the section Supported incremental strategies by adapter.

Custom strategies

> **LIMITED SUPPORT**
>
> Custom strategies are not currently supported on the BigQuery and Spark adapters.

From dbt v1.2 and onwards, users have an easier alternative to creating an entirely new materialization. They define and use their own "custom" incremental strategies by:

1. Defining a macro named `get_incremental_STRATEGY_sql`. Note that `STRATEGY` is a placeholder and you should replace it with the name of your custom incremental strategy.
2. Configuring `incremental_strategy: STRATEGY` within an incremental model.

dbt won't validate user-defined strategies, it will just look for the macro by that name, and raise an error if it can't find one.

For example, a user-defined strategy named `insert_only` can be defined and used with the following files:

macros/my_custom_strategies.sql

```
{% macro get_incremental_insert_only_sql(arg_dict) %}

  {% do return(some_custom_macro_with_sql(arg_dict["target_relation"],
arg_dict["temp_relation"], arg_dict["unique_key"], arg_dict["dest_columns"],
arg_dict["incremental_predicates"])) %}

{% endmacro %}


{% macro some_custom_macro_with_sql(target_relation, temp_relation,
unique_key, dest_columns, incremental_predicates) %}

    {%- set dest_cols_csv = get_quoted_csv(dest_columns |
map(attribute="name")) -%}

    insert into {{ target_relation }} ({{ dest_cols_csv }})
```

```
    (
        select {{ dest_cols_csv }}
        from {{ temp_relation }}
    )

{% endmacro %}
```

models/my_model.sql

```
{{ config(
    materialized="incremental",
    incremental_strategy="insert_only",
    ...
) }}

...
```

If you use a custom microbatch macro, set a `require_batched_execution_for_custom_microbatch_strategy` behavior flag in your `dbt_project.yml` to enable batched execution of your custom strategy.

Custom strategies from a package

To use the `merge_null_safe` custom incremental strategy from the `example` package:

- Install the package
- Add the following macro to your project:

macros/my_custom_strategies.sql

```
{% macro get_incremental_merge_null_safe_sql(arg_dict) %}
    {% do return(example.get_incremental_merge_null_safe_sql(arg_dict)) %}
{% endmacro %}
```

Was this page helpful?

Yes No

Privacy policy Create a GitHub issue

Edit this page

*Last updated on **May 4, 2026***